

RADIOGRAFÍA FORENSE 360° — Antigravity OS / Radar

Formulador 360

Fecha de auditoría: 2026-08-07

Auditor: Chief Software Auditor / DevSecOps Lead / System Architect (inspección en disco, no invasiva)

Alcance: proyecto raíz `c:\2026 AI EGIOC5\Antigravity JS` (servicio Render `radar-formulador-360`). Excluye explícitamente los subproyectos hermanos en `proyectos/` (`Proy_03_RadarFondos`, `Proy_04_Geomatrix`, `Proy_05_SIG`, `api-usuarios`, `react-basico`) — cada uno tiene su propio repositorio git y está fuera del árbol versionado de este proyecto (`.gitignore:19-24`).

Antecedente directo: `docs/RADIOGRAFIA_FORENSE_360_2026-08-06.md` (auditoría de ayer) → `docs/analisis_gaps_v1.md` (plan de remediación) → `docs/ESTADO.md` (bitácora de ejecución, Oleadas 0-3 cerradas). Esta radiografía es una verificación independiente, hecha leyendo el código actual sin asumir que la bitácora está actualizada, y confirma o corrige lo que esa bitácora declara.

Nota de conservación (2026-08-07, post-remediación): este documento es el snapshot ANTERIOR a la Operación Exterminio Final (purga de `pg`, `EGIOC5/`, `OPENCODE-MODEL/`, `MiniMaxChat.jsx` original, guardrail RLS, corrección del gate de arquitectura). Se conserva como evidencia histórica del estado pre-remediación. El estado posterior está en `docs/INFORME_RECONCILIACION_CIERRE_2026-08-07.md` y en la radiografía re-emitida más reciente.

1. TOPOGRAFÍA DE ARQUITECTURA Y PATRÓN DE DISEÑO

Stack real (verificado en `package.json` e imports reales, no en documentación)

Capa	Tecnología	Evidencia
Frontend	React 18.3.1 + React Router 7.16 + Vite 5.3.4 + Tailwind CSS 4.3	<code>package.json:35-37</code> , <code>vite.config.js:37</code> (<code>root:'public'</code>)
Backend	Node.js 20+ ESM + Express 4.22.1, proceso único monolítico	<code>server.js</code> (456 líneas, un solo archivo de arranque)
Tiempo real	WebSocket nativo (<code>ws</code> 8.21.0), path <code>/ws/live_radar</code>	<code>server.js:375-390</code>
Base de datos	Supabase PostgreSQL vía REST/PostgREST , no driver <code>pg</code> directo	<code>src/modules/formulador/supabaseClient.js</code> (fetch crudo a <code>/rest/v1</code>)
Auth	Firebase Authentication (Google Sign-In únicamente) + capa JWT propia complementaria	<code>public/src/lib/firebase.js</code> , <code>src/shared/infrastructure/session-manager.js</code>
IA — motor principal	Claude / Anthropic SDK (<code>claude-sonnet-4-6</code>)	<code>server.js:29</code> , <code>src/modules/radar/m1Pipeline.js:13</code>
IA — búsqueda	Tavily Search API	<code>src/modules/radar/m1Pipeline.js:51-77</code>

Capa	Tecnología	Evidencia
web		
Caché	Upstash Redis (REST, sin SDK) con fallback a Map en memoria	src/shared/infrastructure/cache.js
Email transaccional	Brevo (antes Sendinblue)	src/modules/communications/infrastructure/BrevoEmailAdapter.js

Hallazgo — dependencia muerta: pg (^8.21.0) está declarado en package.json:34 pero no se importa en ningún archivo del árbol (grep "from 'pg'" → 0 resultados). El acceso real a PostgreSQL es 100% vía REST de Supabase (supabaseClient.js). Esto es ruido de dependencias, no una integración real — coherente con el comentario de render.yaml:21-23 , que ya lo advierte explícitamente.

Patrón arquitectónico

Monolito modular con capas ligeras estilo hexagonal solo en el módulo de comunicaciones. No es DDD puro ni hexagonal estricto en todo el árbol:

- src/modules/communications/ sí separa domain/ (EmailSender.js , interfaz abstracta), application/ (SendEmailUseCase.js) e infrastructure/ (BrevoEmailAdapter.js , CommunicationRouter.js) — el único módulo con desacoplamiento real de puerto/adaptador.
- src/modules/formulador/ es más pragmático: FormuladorRouter.js → FormuladorPgController.js → supabaseClient.js , sin capa de dominio explícita; la lógica de negocio (tenant, UUID, RLS) vive directamente en el controller (FormuladorPgController.js:12-39).
- src/modules/radar/m1Pipeline.js mezcla en un solo archivo: definición de tool-use de Claude, llamada REST a Tavily, lógica de caché, rutas Express y el loop agéntico — sin separación de capas.
- server.js es el composition root: importa los 4 routers de módulo y los monta directamente, sin capa de aplicación intermedia.

Nivel de acoplamiento: **medio-alto entre Presentación (rutas Express) y Aplicación** — los controllers llaman directo a supabaseClient / Anthropic SDK sin interfaces intercambiables (excepto Communications). Aceptable para el tamaño actual del sistema, pero no es Hexagonal/DDD en el sentido estricto que declaraba AGENTS.md:33 ("001_ARQUITECTO_CORE: Define patrones (Hexagonal, DDD)") — esa declaración era aspiracional, no descriptiva del código real (corregida en la remediación del mismo día).

Manejo de estado y SPOF

Estado	Dónde vive	Persiste reinicio	Riesgo
radarData (convocatorias seed + feed live)	Variable let en memoria de server.js:57	✗ No — se resetea al seed hardcodeado en cada restart	SPOF: un solo proceso Node sostiene todo el estado del Radar; sin réplica, un crash pierde el feed en vivo hasta el próximo ciclo de cron
Sesiones JWT propias	cache.js → Upstash Redis si está configurado, si no Map en memoria	✓ Con Redis / ✗ sin Redis	Hoy Redis Sí está configurado (UPSTASH_REDIS_REST_URL presente en .env:21), así que las sesiones sobreviven a un restart
Cuota diaria por IA (checkQuota)	Map en memoria, session-manager.js:76	✗ No	Deliberado según comentario del propio código (session-manager.js:73-74): "acceptable para evitar abuso, no facturación exacta"

Estado	Dónde vive	Persiste reinicio	Riesgo
Rate limit por ráfaga (<code>checkBurst</code>)	<code>Map</code> en memoria, <code>session-manager.js:99</code>	✗ No, por diseño	Ventana de 10s, sin necesidad de persistir
Datos de negocio (proyectos Formulador, Módulo 10)	Supabase PostgreSQL real	✓ Sí	Única fuente de verdad persistente del sistema
Checkpoints de estado (<code>StateManager.js</code>)	Archivos JSON locales en <code>logs/checkpoints/</code>	✓ En disco local del proceso	SPOF real: si Render reasigna el contenedor (deploy free tier, sin volumen persistente), estos checkpoints desaparecen — no hay evidencia de que se usen en el flujo actual (<code>stateManager</code> no se importa en <code>server.js</code> ni en ningún router)

SPOF crítico identificado: todo el backend es **un único proceso Express en un único dyno de Render (plan free)** (`render.yaml:8`). No hay balanceo, ni réplica, ni cola de trabajos. El WebSocket de Radar (`server.js:375`) vive en memoria de ese mismo proceso — si cae, todos los clientes conectados pierden el feed hasta reconexión (mitigado client-side con retry de 4s, `RadarApp.jsx:15`).

2. INVENTARIO DE MVP Y FUNCIONALIDADES (REAL VS. STUBS)

Rutas de interfaz (React Router, `public/src/App.jsx:13-36`)

Ruta	Módulo	Estado real	Evidencia
<code>/inicio</code>	Login	● Real — Firebase Google Sign-In funcional	<code>InicioPage.jsx:17-30</code>
<code>/radar</code>	A · Monitoreo	● Real — REST + WebSocket en vivo, búsqueda IA on-demand (Claude+Tavily)	<code>RadarApp.jsx</code> , <code>server.js:95-98,201,375-437</code>
<code>/panel</code>	A · Monitoreo	● Stub puro — <code>FrozenPage.jsx</code> genérico, sin datos, con badge "Frozen — Próximamente"	<code>FrozenPage.jsx:1-31</code> , referenciado en <code>App.jsx:22</code>
<code>/directorío</code>	A · Monitoreo	● Stub puro	Idéntico patrón <code>FrozenPage</code>
<code>/favoritos</code>	A · Monitoreo	● Stub puro — requiere modelo de datos nuevo, no existe tabla <code>favoritos</code> en las migraciones	<code>docs/analisis_gaps_v1.md:38</code>
<code>/calendario</code>	A · Monitoreo	● Stub puro — depende de Favoritos	Idéntico patrón <code>FrozenPage</code>
<code>fase1-entrada.html</code>	B · Formulación	● Real — fuera del SPA React (HTML standalone servido por Express estático), persiste Módulos 1-9 vía <code>POST /api/formulador/fase1</code>	<code>FormuladorRouter.js:12</code> , confirmado operativo en <code>docs/ESTADO.md:24-33</code>
<code>/modulo10</code>	B · Formulación	● Real — única pantalla de Formulador ya migrada al SPA React, CRUD completo contra Supabase	<code>Modulo10Page.jsx</code> (169 líneas, fetch real con token Firebase)
<code>/anexos</code>	B · Formulación	● Stub puro	<code>FrozenPage</code>

Ruta	Módulo	Estado real	Evidencia
/logistica	B · Formulación	● Stub puro	FrozenPage
/dialetica	B · Formulación	● Stub puro	FrozenPage
/ficha	B · Formulación	● Stub puro en el SPA — pero el motor que la alimentaría (<code>orchestrator-engine.js</code>) Sí es real y Sí está conectado a un endpoint (<code>POST /api/formulador/ficha-tecnica</code>), solo que aún no tiene pantalla propia	<code>FormuladorPgController.js:177-203</code> , <code>docs/ESTADO.md:31</code>

Resumen MVP: de 11 rutas declaradas en el Sidebar, **3 están completamente operativas** (`/radar` , `/modulo10` , `fase1-entrada.html`), **1 tiene motor real sin pantalla** (`/ficha`), y **6 son maquetas visuales sin ningún dato ni lógica** (todas usan el mismo componente `FrozenPage.jsx` , que ni siquiera intenta un fetch — es honesto sobre su propio estado, con el badge "Frozen").

Motor multiagente Orchestrator000 (`src/orchestrator-engine.js`) — real vs. plantilla

Este es el hallazgo más matizado de la auditoría. El "sistema multiagente" que redacta la Ficha Técnica tiene 4 agentes con niveles de "inteligencia" muy distintos:

- **AGT-052 (Administrativo/Legal)** — ● el único que llama realmente a Claude (`callAI()` , `orchestrator-engine.js:114`), con fallback a texto de plantilla si la llamada falla.
- **AGT-053 (Operativo/Financiero)** — ● no usa IA en absoluto: es aritmética determinista (AIU 25% + IVA 19% sobre AIU, `orchestrator-engine.js:142-145`). Funcional y correcto, pero no es "IA" pese a estar clasificado como agente.
- **AGT-054 (Riesgos)** — ● lista de riesgos **hardcodeada** (4 riesgos fijos, con probabilidad condicionada por 2-3 banderas booleanas de la ficha) — no genera contenido dinámico ni usa IA (`orchestrator-engine.js:163-198`).
- **AGT-056 (Evaluador)** — ● checklist de 8 reglas booleanas determinista (`orchestrator-engine.js:223-265`), sin IA. Actúa además como **gate de arquitectura real**: `Orchestrator000.run()` exige `disenoAprobado.aprobado === true` y verifica un hash FNV-1a de la ficha antes de ejecutar (`orchestrator-engine.js:328-339`) — mecanismo anti-manipulación genuino, no cosmético.

Conclusión: el pipeline es real y produce resultados coherentes, pero de 4 "agentes" solo 1 usa un LLM; los otros 3 son lógica de negocio determinista etiquetada como agente.

agents/ (carpeta raíz, 22 subcarpetas) — confirmado no-runtime de negocio (matizado el 2026-08-07)

`scripts/generar_reporte.cjs:18-21` lo declara explícitamente: "esto es una librería de prompts/skills para uso interactivo, no un runtime — ningún agente de `agents/` corre como proceso en `server.js`". Verificado: `server.js` no importa nada de `agents/`. El inventario real generado en vivo (`public/estado_antigravity.json`) muestra **8 de 15 carpetas "definidas"** y **7 solo con `IDENTITY.md`** sin ningún skill implementado (`005_Radar1_minero` , `006_Radar2_Estratega` , `015_intelligence-core` , `03-analista-secop` , `07-ing-concreto_GFRC` , `08-estratega-neuromarketing` , `14-analista-comportamiento`).

Matiz descubierto en la remediación de este mismo día (ver `docs/INFORME_RECONCILIACION_CIERRE_2026-08-07.md`): `agents/000_Orquestador.cjs` sí es un script real y ejecutable (CLI, no wireado a `server.js`) que corre agentes como child process y aplica un gate de arquitectura. No contradice el hallazgo anterior (no corre en el request path del servidor web), pero matiza "no-runtime" — es runtime de herramienta de desarrollo, no de negocio.

3. MÓDULO DE ADMINISTRACIÓN Y SEGURIDAD (RBAC Y PERÍMETRO)

Panel de administración

● **No existe.** Búsqueda exhaustiva de `/admin` en todo el árbol de código (`.js` / `.jsx` / `.cjs`) → **0 resultados**. `docs/ESTADO.md:54-57` lo confirma como "Oleada 4 — Panel `/admin` mínimo viable", con sus 3 checkboxes sin marcar.

RBAC — qué existe realmente

- El campo `role` viaja en el JWT propio (`session-manager.js:22-30`), default `'user'` .
- El único lugar del sistema que lee `role === 'admin'` es `revokeSession()`** (`session-manager.js:47`).
- No hay middleware `requireAdmin` reutilizable (`docs/analisis_gaps_v1.md:53-55` , brecha C1 sin resolver).
- Ningún endpoint de negocio (Radar, Formulador, Comunicaciones, GitHub) está protegido por rol — todos exigen solo *estar autenticado*, no un rol específico.
- El frontend (`Sidebar.jsx`) no tiene ninguna sección condicionada por `user.role` .

Perímetro de autenticación (`server.js:78-91`)

Gate real y correctamente implementado:

```
PUBLIC_API_PREFIXES = ['/api/health', '/api/convocatorias', '/api/session']
```

Todo lo demás bajo `/api/` pasa por `verifyFirebaseAuth` (`FirebaseAuthMiddleware.js:3-17`) — **fail-closed**: si `serviceAccountKey.json` no carga, el propio código lo documenta (`FirebaseAdmin.js:22`): "sin credenciales válidas: `verifyIdToken()` fallará y el middleware rechazará TODAS las requests, no hay bypass". Diseño de seguridad correcto.

Después del gate corre `burstLimiter` sobre todo `/api/*` (`server.js:90`), 20 req/10s por `uid` o IP.

No se encontraron endpoints con lógica de negocio sensible fuera del gate. El perímetro está bien cerrado para lo que existe hoy — el vacío no es una fuga, es la ausencia total de un panel privilegiado.

Aislamiento multi-tenant

Real pero no es RLS-por-rol como se documentaba en `AGENTS.md` . `supabaseClient.js:36-52` confiesa en su propio comentario: **Firestore no está configurado como Third-Party Auth en el dashboard de Supabase, así que el JWT de usuario final es rechazado por PostgREST (401 `PGRST301`)** y todas las llamadas degradan silenciosamente a `SERVICE_KEY` , que **bypasea RLS por completo**. El aislamiento real hoy lo da el filtro explícito `WHERE tenant_id = p_tenant_id` dentro de cada función RPC de Postgres (tenant derivado determinísticamente del UID de Firebase vía SHA-256). Funciona, pero un error futuro en cualquier función RPC que olvide ese `WHERE` rompería el aislamiento sin que RLS lo detenga. (Actualización 2026-08-07: se agregó un guardrail duro en `supabaseClient.js:rpc()` que aborta en Node si `p_tenant_id` es nulo/inválido antes de tocar Postgres — ver informe de reconciliación.)

Hallazgo crítico de seguridad — secretos en texto plano

`.env` contiene en texto plano: clave de Anthropic, Tavily, Supabase (anon + **service key**, con privilegios de bypass total de RLS), JWT secret, token de GitHub, API key de Render, Stitch, OpenRouter, Groq y Gemini. **Correctamente excluido de git** (verificado con `git ls-files` → no trackeado; `git log --all -- .env` → vacío, nunca commiteado). El riesgo es exclusivamente de exposición local/filesystem, no de fuga en el repositorio remoto. Pendiente real y no resuelto: `docs/ESTADO.md:14` documenta una `SUPABASE_SERVICE_KEY` huérfana aún activa (200 OK) sin revocar, bloqueada porque requiere acción manual en el dashboard de Supabase.

4. SISTEMA MULTIAGENTE, LLMs Y FINOPS (CONTROL DE TOKENS)

Puntos de integración LLM reales

Motor	Uso real	Endpoint(s)	Estado
Claude / Anthropic (<code>claude-sonnet-4-6</code>)	Motor principal — chat proxy, pipeline Radar (tool-use con Tavily), AGT-052	<code>POST /api/chat</code> , <code>POST /api/minimax/chat</code> (alias legacy), <code>/api/radar/search</code> , <code>/api/radar/stream</code> , <code>orchestrator-engine.js</code>	● Operativo (código); ● saldo de cuenta Anthropic agotado, ver informe de reconciliación 2026-08-07 \$1
Tavily Search	Búsqueda web en vivo, tool nativa de Claude	<code>m1Pipeline.js:51-77</code> , dominios <code>.gov.co</code> en whitelist	● Operativo
OpenRouter	Fallback declarado "cuando Anthropic sin créditos"	<code>GET /api/openrouter/models</code>	<code>status</code> , <code>POST /api/openrouter/chat</code> ● Código real y probado, pero sin ningún caller interno — es un proxy expuesto, no un fallback automático real (reconectado a MiniMaxChat.jsx el 2026-08-07)
MiniMax	Etiqueta de UI únicamente — <code>MiniMaxChat.jsx</code> llama a <code>/api/minimax/chat</code> , que en <code>server.js:139-168</code> es literalmente el mismo cliente Claude	<code>MiniMaxChat.jsx:17,45</code>	● Nombre engañoso — funcionalmente real (es Claude), pero el branding "MiniMax M2.5 AI" no corresponde al motor que responde. Corregido 2026-08-07: reconectado a <code>/api/openrouter/chat</code> con modelo real <code>minimax/minimax-m2.5</code> , verificado contra la lista pública de OpenRouter.
Groq, Gemini	Claves presentes en <code>.env:78-79</code>	Ningún import en el backend	● Standby — el propio <code>.env</code> las etiqueta: "no usadas por el backend, solo por <code>.kilo/IDE extension</code> "

FinOps — control de consumo de tokens

- **Captura:** `AuditLogger.log('CLAUDE_CHAT_SUCCESS', { model, tokens: response.usage })` — sí se registra el consumo por request, con doble respaldo (archivo local `logs/audit.log` append-only + colección Firestore `audit_logs`).
- **Agregación / alertas / dashboard:** ● **ausente.** No existe ningún endpoint que sume tokens por usuario/día, ni alerta de costo (`docs/analisis_gaps_v1.md:65-67` , brecha D1 resuelta solo en el plan, no en el código).

- **Integración con Langfuse/Helicone:** ● ausente, decisión explícita de no incorporar herramientas de terceros con costo recurrente.
- **Control de gasto real implementado:** `checkQuota()` limita a 50 consultas de IA por `uid` /día — guardrail de abuso, no FinOps real (no calcula costo en dinero, se resetea en memoria).
- **Evidencia de volumen real de uso:** `logs/audit.log` contenía **una sola línea** al momento de esta auditoría (`SESSION_LOGIN` de un uid de prueba) — el sistema estaba en fase de pruebas, sin tráfico de producción acumulado todavía.
- **Hallazgo posterior (2026-08-07, mismo día):** el `GET /api/health` reporta `claude: ✓ configurado` solo verificando que la variable de entorno exista, nunca probando la API real. Se descubrió en vivo que la cuenta de Anthropic tiene saldo agotado — un falso positivo de salud que el propio FinOps ausente no puede detectar. Ver `docs/INFORME_RECONCILIACION_CIERRE_2026-08-07.md §1`.

Orquestadores de agentes

No hay un orquestador de procesos multiagente corriendo en producción (request path del servidor web). La única orquestación real en runtime de negocio es `Orchestrator000`, que corre 3 de sus 4 "agentes" en paralelo vía `Promise.all` dentro de una sola request HTTP. Aparte, `agents/000_Orquestador.cjs` es un orquestador real pero de herramienta de desarrollo (CLI), no de negocio — ver §2.

5. TELEMETRÍA, MONITORIZACIÓN Y MODOS STANDBY (ENV FLAGS)

SDKs de monitoreo de terceros

- **Ausentes por completo.** Búsqueda de Sentry/PostHog/Google Analytics/ `gtag` en todo el código fuente → **0 coincidencias reales de uso**. La única observabilidad es `console.log` / `console.error` y el `AuditLogger` custom.

Variables de entorno en "modo standby"

No existe el patrón literal `ENABLED=false`, pero sí standby por **ausencia de valor**:

Variable	Estado en <code>.env</code>	Efecto
<code>DATABASE_URL</code>	Vacía	Confirma que la app no usa conexión <code>pg</code> directa
<code>STRIPE_SECRET_KEY</code>	~~Vacía~~ Eliminada de <code>.env</code> el 2026-08-07 (placeholder sin SDK ni consumidor)	Monetización inactiva
<code>INNGEST_EVENT_KEY</code>	Vacía	Sin jobs/colas Inngest
<code>MCP_API_KEY</code> , <code>RESEND_API_KEY</code>	Vacías	Sin integraciones correspondientes
<code>BREVO_SENDER_EMAIL</code>	Vacía a propósito	Bloquea en runtime <code>POST /api/communications/email</code> — el adaptador está completo, pero sin remitente verificado en Brevo la llamada real fallará
<code>GROQ_API_KEY</code> , <code>GEMINI_API_KEY</code>	Presentes con valor	Standby por diseño — sin consumidor en el backend
<code>NODE_ENV</code>	<code>development</code> local	<code>render.yaml:11</code> fija <code>production</code> para el deploy — divergencia correcta y esperada

6. MONETIZACIÓN, PASARELAS Y MODELO DE NEGOCIO (SaaS)

Esquema de base de datos

Las 6 migraciones SQL existentes definen exclusivamente tablas de dominio del Formulador (proyectos, fase1, indicadores Módulo 10) con `tenant_id` como columna de aislamiento. **No existe ninguna tabla `users`, `subscriptions`, `plans`, `invoices` ni manejo de moneda/planes**. El concepto de "tenant" hoy es 1:1 con un usuario individual de Firebase — no existe la noción de "organización" o "equipo" en el esquema.

Pasarelas de pago

● **Completamente ausente — ni siquiera en borrador**. Búsqueda de Stripe/Wompi/Bold/Mercado Pago/webhooks de facturación en todo el árbol de código → sin resultados relevantes. `STRIPE_SECRET_KEY` (ahora eliminada de `.env`) nunca tuvo SDK asociado en `package.json`. No hay webhook de facturación recurrente en ningún router.

Moneda

El único tratamiento de moneda es funcional dentro del motor de Ficha Técnica: AGT-053 calcula presupuestos en COP implícitamente (AIU 25% + IVA 19%), consistente con el Axioma II de `AGENTS.md:23` ("Soberanía Financiera Absoluta... COP"). Esto es cálculo de presupuesto de proyecto, no un sistema de facturación SaaS.

Decisión de producto confirmada, no un olvido: `docs/analisis_gaps_v1.md:75-79` documenta que la monetización fue evaluada y pospuesta explícitamente por el usuario el 2026-08-06, en espera de una base de usuarios que la justifique.

7. MATRIZ DE DIAGNÓSTICO DEFINITIVA (snapshot 2026-08-07, pre-remediación)

#	Módulo / Subsistema	Estado	Evidencia clave
1	Autenticación (Firebase Google Sign-In)	● OPERATIVO	<code>InicioPage.jsx</code> , <code>FirebaseAuthMiddleware.js</code>
2	Gate universal <code>/api/*</code> (perímetro auth)	● OPERATIVO	<code>server.js:78-91</code>
3	Sesión JWT propia + revocación	● OPERATIVO	<code>session-manager.js:22-56</code>
4	Rate limiting por ráfaga	● OPERATIVO	Verificado con 25 requests reales (<code>docs/ESTADO.md:17</code>)
5	Validación de schema (zod) en POST	● OPERATIVO	<code>validation.js</code> , 4 endpoints
6	Módulo A — Radar (REST + WS + IA)	● OPERATIVO (código)	<code>RadarApp.jsx</code> , <code>m1Pipeline.js</code> — bloqueado en la práctica por saldo Anthropic
7	Feed "Live" del WebSocket de Radar	● OPERATIVO (código)	Cron real cada 6h — mismo bloqueo de saldo
8	Módulo B — Fase 1 (Módulos 1-9)	● OPERATIVO	<code>FormuladorRouter.js:12</code> , persistencia Supabase
9	Módulo B — Módulo 10 (Indicadores)	● OPERATIVO	<code>Modulo10Page.jsx</code> , migración desplegada

#	Módulo / Subsistema	Estado	Evidencia clave
10	Motor Orchestrator000 (AGT-052/053/054/056)	● OPERATIVO (backend)	Gate de hash real, conectado a endpoint
11	Pantalla <code>/ficha</code> en el SPA	● INCOMPLETO / STUB	Motor real sin consumidor visual
12	Módulo A — Panel/Directorio/Favoritos/Calendario	● AUSENTE	4× <code>FrozenPage.jsx</code>
13	Módulo B — Anexos/Logística/Dialéctica	● AUSENTE	3× <code>FrozenPage.jsx</code>
14	Comunicaciones (email Brevo)	● STANDBY	Adaptador completo, bloqueado por sender vacío
15	Conector GitHub	● OPERATIVO	Token corregido 2026-08-06, 200 OK verificado
16	Proxy OpenRouter	● OPERATIVO	Reconectado a MiniMaxChat.jsx el 2026-08-07, modelo verificado
17	Motores Groq / Gemini	● STANDBY	Claves presentes, cero imports en backend
18	"MiniMax M2.5 AI" (branding)	● OPERATIVO (corregido)	Reconectado a OpenRouter real 2026-08-07, ya no es alias de Claude
19	Panel <code>/admin</code>	● AUSENTE	0 coincidencias en todo el árbol
20	Middleware <code>requireAdmin</code> / RBAC granular	● AUSENTE	Solo se lee <code>role</code> en <code>revokeSession()</code>
21	Aislamiento multi-tenant (RLS por rol)	● INCOMPLETO (mitigado)	Degrada a <code>SERVICE_KEY</code> ; guardrail duro agregado 2026-08-07 en <code>supabaseClient.js</code>
22	FinOps — captura de tokens	● OPERATIVO	<code>AuditLogger</code> registra <code>usage</code> por request
23	FinOps — dashboard/alertas agregadas	● AUSENTE	Sin endpoint de agregación — urgencia elevada tras hallazgo de saldo agotado
24	Telemetría (Sentry/PostHog/GA)	● AUSENTE	Pospuesto por decisión explícita
25	Pasarela de pago / suscripciones	● AUSENTE	Sin SDK, esquema ni webhook
26	Esquema de moneda / facturación SaaS	● AUSENTE	Solo presupuesto de proyecto, no facturación de plataforma
27	Caché dual (Redis Upstash + memoria)	● OPERATIVO	Redis configurado y activo hoy
28	<code>agents/</code> (22 carpetas) como runtime de negocio	● AUSENTE (por diseño)	Confirmado no-runtime de negocio; sí existe runtime de dev-tool (<code>000_Orquestador.cjs</code>)
29	Reporte de estado de agentes	● OPERATIVO	Se regenera cada 10 min, datos verificables
30	Dependencia <code>pg</code> (driver directo)	✅ RESUELTO 2026-08-07	<code>npm uninstall pg</code> ejecutado
31	<code>StateManager.js</code> (checkpoints locales)	● INCOMPLETO	Funcional pero sin caller en el flujo actual
32	Firestore Security Rules (<code>audit_logs</code>)	● OPERATIVO	Acceso restringido al propietario, deny-all por defecto
33	Secretos en <code>.env</code> (higiene)	● STANDBY (con pendiente)	Nunca commiteado; <code>SUPABASE_SERVICE_KEY</code> huérfana sin revocar; <code>STRIPE_SECRET_KEY</code> eliminada
34	<code>EGIOC5/</code> , <code>OPENCODE-MODEL/</code> (dirs raíz)	✅ RESUELTO 2026-08-07	Carpetas vacías, purgadas

#	Módulo / Subsistema	Estado	Evidencia clave
35	Gate de arquitectura (<code>agents/000_Orquestador.cjs</code>)	● OPERATIVO (corregido 2026-08-07)	Reemplazado el autofirmado falso por veredicto real vía <code>.claude/agents/architect.md</code>
36	Anthropic API — saldo de cuenta	● CRÍTICO	Descubierto en vivo 2026-08-07, bloquea items 6, 7, 10, 16 en la práctica

Conclusión ejecutiva

El sistema tiene un **núcleo real y verificado en producción**: autenticación, perímetro de seguridad, Radar (Módulo A núcleo) y Formulador Fase 1 + Módulo 10 (Módulo B núcleo) funcionan con datos reales, persistencia real en Supabase, y un motor de IA (Claude + Tavily) genuinamente integrado con function-calling, no simulado — aunque bloqueado hoy por saldo de cuenta agotado. La disciplina de ingeniería es alta: hay gates de arquitectura reales, rate limiting probado con requests HTTP reales, y una bitácora de decisiones (`docs/ESTADO.md`) que documenta con honestidad qué quedó pendiente y por qué.

Los vacíos no son deuda técnica oculta — son **decisiones explícitas y documentadas** de posponer 3 frentes completos (RBAC/panel admin, telemetría de terceros, monetización) a favor de cerrar primero el flujo end-to-end del Formulador. Ver `docs/INFORME_RECONCILIACION_CIERRE_2026-08-07.md` para el detalle completo de la remediación ejecutada el mismo día y el hallazgo crítico de saldo de Anthropic.

Radiografía generada por inspección directa de archivos reales — cada hallazgo cita ruta y, cuando aplica, número de línea.